

復習

目次

復習の前に

if文とは
for文とは
配列とは

いろいろな使用例

- ・if文 -数学編-
- ・if文 -文字列編-
- ・for文 -配列処理-

プログラムを見て処理を把握しよう

復習の前に

バンタンの生徒を大きくグルーピングすると

- ・ なにか作ろうと思ったらAiに頼りきりになる人
- ・ 簡単なif文やUnityの機能程度は扱える人 パイザCD
- ・ for文配列等の処理が難なく書ける人 パイザBC ギリA
- ・ 授業の内容は完璧に理解し効率的なプログラムを追い求める変な人 パイザSA

if文

別名を条件分岐といいます。
条件を満たした場合のみ実行するプログラムを作成することで、
ユーザーの行動によって処理を変えたり、
設定によって処理を変えたりできます。

if文

```
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20
```

0 個の参照

```
static void Main(string[] args)  
{  
    int number = 1;  
  
    if(number ==1)  
    {  
        Console.WriteLine("Numberは1です。");  
    }  
    else  
    {  
        Console.WriteLine("Numberは1以外です。");  
    }  
}
```

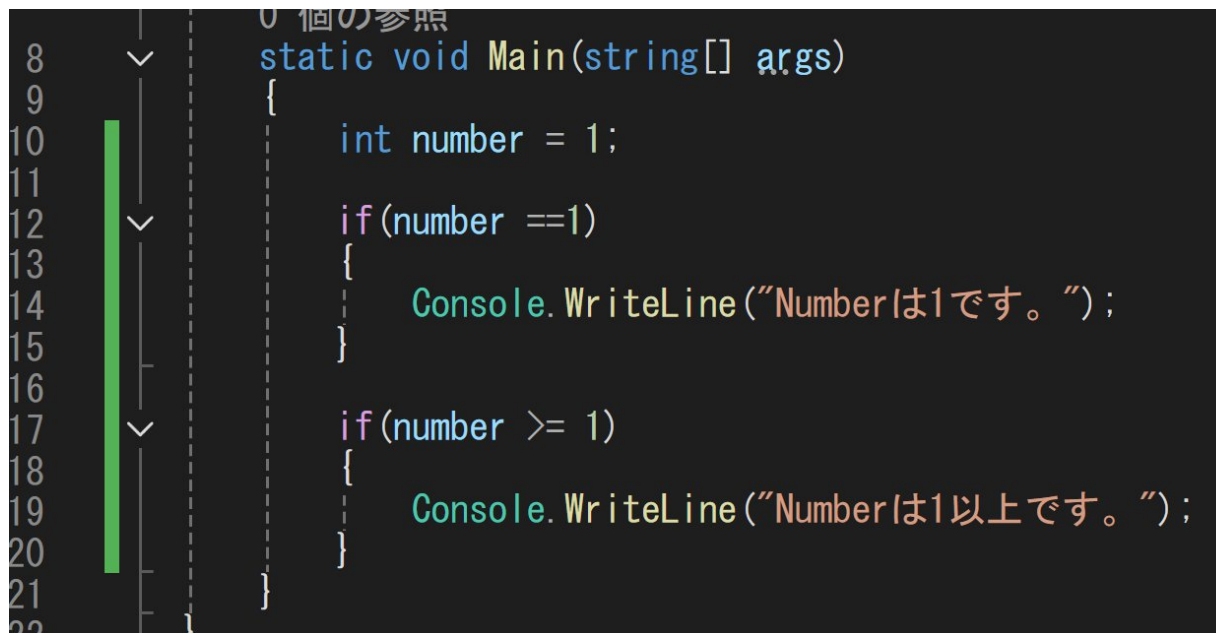
if文

解説

if(条件)の比較の仕方はいくつかあります。

これ以外にもありますが、下記が代表的なものです。

- == 中身が同じ
- != 中身が違う
- >= 左側が右側以上
- <= 左側が右側以下
- > 左側が右側より大きい
- < 左側が右側より小さい



```
8  static void Main(string[] args)
9  {
10     int number = 1;
11
12     if(number ==1)
13     {
14         Console.WriteLine("Numberは1です。");
15     }
16
17     if(number >= 1)
18     {
19         Console.WriteLine("Numberは1以上です。");
20     }
21 }
22
```

複雑な条件＜連続したif文＞

解説

if文は連続して書くことができます。
書くことはできますが、長くなれば長くなるほど分かりにくくなるので、
出来る限り避けるべきではありません。

実行例

```
6 void main()
7 {
8     int number = 1;
9     string plusText = "正の数です。";
10    string minusText = "負の数です。";
11    string oneText = "1です";
12    string otherText = "それ以外です。";
13
14    if (number > 0)
15    {
16        printf(plusText.c_str());
17    }
18    else if (number < 0)
19    {
20        printf(minusText.c_str());
21    }
22    else if (number == 1)
23    {
24        printf(oneText.c_str());
25    }
26    else
27    {
28        printf(otherText.c_str());
29    }
30 }
31
```

複雑な条件<かつ、または(論理積、論理和)>

解説

条件は複雑にすることができます。
これも、必要以上に複雑になる可能性があるので、
極力使わないようにする事が多いです。

実行例

```
6 void main()
7 {
8     int number = 1;
9     string oneUpThreeDownText = "1以上 3 以下です。";
10    string fourOrFive = "4か5です。";
11    string otherText = "それ以外です。";
12
13    if (number >= 1 && number <= 3)
14    {
15        printf(oneUpThreeDownText.c_str());
16    }
17    else if( number == 4 || number == 5)
18    {
19        printf(fourOrFive.c_str());
20    }
21    else
22    {
23        printf(otherText.c_str());
24    }
25 }
```

複雑な条件<if文のネスト(2重構造)>

解説

if文の中にもif文を書く事ができます。
これも複雑になるので余り推奨はされません。

実行例

```
6 void main()
7 {
8     int number = 1;
9     string zeroUpText = "正の数です";
10    string oneText = "1です。";
11
12    if (number > 0)
13    {
14        printf(zeroUpText.c_str());
15
16        if (number == 1)
17        {
18            printf(oneText.c_str());
19        }
20    }
21 }
```

for文

別名を繰り返し処理といいます。
指定の回数繰り返す処理を記述したり、
配列の中身すべてに何かしたい場合に使います。

for文

```
0 個の参照
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine(i + "回目");
        }
    }
}
```

for文

```
for (int i = 0; i < 5; i++)  
{  
    Console.WriteLine(i + "回目");  
}
```

for文の()の中は3つの要素があります。

赤: 何から始めるから → 0

青: 何まで行うか → 5

緑: 1ステップどこまで進むか → +1

例だと 0から4まで1ずつ繰り返す文になっています。

0回目 1回目 2回目 3回目 4回目

配列とは

配列とは、同じ種類のデータをまとめて管理するための仕組みです。
今までは一つの変数に一つのデータを入れていましたが、
配列を使う事で複数のデータをまとめて入れることができます。

配列を使う事で同じ意味を持つデータをまとめて扱う事ができます。

イメージ

```
int number;
```



```
int number[10];
```



※ 配列は0番目から始まるので注意が必要です。

配列の作り方

```
// 配列の宣言のしかた
```

```
//要素数が分かるけど、中身が未確定の場合
```

```
int count = 10;
```

```
int[] numbers = new int[count];
```

```
// 中身が何か確定している場合
```

```
int[] numbers2 = {0, 1, 2, 3, 4, 5, 6 };
```

配列の作り方①

```
//要素数が分かるけど、中身が未確定の場合  
int count = 10;  
int[] numbers = new int[count];
```

numbersの中身は、、、

数値[0]	数値[1]	数値[2]	数値[3]	数値[4]	数値[5]	数値[6]	数値[7]	数値[8]	数値[9]
0	0	0	0	0	0	0	0	0	0

上記のようにとりあえず全部0(デフォルト)のデータが入る

配列の作り方②

```
// 中身が何か確定している場合  
int[] numbers2 = {0, 1, 2, 3, 4, 5, 6};
```

numbersの中身は、、、

数値[0]	数値[1]	数値[2]	数値[3]	数値[4]	数値[5]	数値[6]
0	1	2	3	4	5	6

上記のようにとりあえず全部0(デフォルト)のデータが入る

配列の使い方

```
int[] numbers = { 3, 4, 7, 5, 1, 2, 3 };  
  
Console.WriteLine(numbers[2]);  
  
int pickUp = numbers[4];  
  
numbers[5] = 10;
```

配列の使い方①

```
int[] numbers = { 3, 4, 7, 5, 1, 2, 3 };  
Console.WriteLine(numbers[2]);
```

numbersの中身は、、

数値[0]	数値[1]	数値[2]	数値[3]	数値[4]	数値[5]	数値[6]
3	4	7	5	1	2	3



Console.WriteLineで [2]を指定している

7の数値が表示される

配列の使い方②

```
int[] numbers = { 3, 4, 7, 5, 1, 2, 3 };  
int pickUp = numbers[4];
```

numbersの中身は、、

数値[0]	数値[1]	数値[2]	数値[3]	数値[4]	数値[5]	数値[6]
3	4	7	5	1	2	3



今回は[4]を指定しているので、、

pickUpには 数値の 1 が保存される

配列の使い方③

```
int[] numbers = { 3, 4, 7, 5, 1, 2, 3 };  
numbers[5] = 10;
```

numbersの中身は、、

数値[0]	数値[1]	数値[2]	数値[3]	数値[4]	数値[5]	数値[6]
3	4	7	5	1	2	3



今回は[5]を指定しているので、、

数値[0]	数値[1]	数値[2]	数値[3]	数値[4]	数値[5]	数値[6]
3	4	7	5	1	10	3

配列の要素数

配列の要素数とは配列に何個データがあるかのことです。
配列の長さ、と言ったりもします。

数値[0]	数値[1]	数値[2]	数値[3]	数値[4]	数値[5]	数値[6]
3	4	7	5	1	2	3



要素数 7

```
static void Main(string[] args)
{
    int[] scores = { 80, 75, 90 };
    int count = scores.Length;
}
```

for文と配列

配列はfor文と組み合わせる事でその本領を発揮します。
特に配列全体に対して何かをする際には必ずと言っていいほど使います。

例えば先ほど書いたこのプログラムは配列すべてを表示しています。

```
int[] scores = { 80, 75, 90 };  
  
Console.WriteLine(scores[0]);  
Console.WriteLine(scores[1]);  
Console.WriteLine(scores[2]);
```

これを書き直すとこのようになります。

```
int[] scores = { 80, 75, 90 };  
  
for (int i = 0; i < scores.Length; i++)  
{  
    Console.WriteLine(scores[i]);  
}
```

for文と配列

動きとしては、

```
int[] scores = { 80, 75, 90 };  
for (int i = 0; i < scores.Length; i++)  
{  
    Console.WriteLine(scores[i]);  
}
```

数値[0]	数値[1]	数値[2]
80	75	90

for文 1週目 $i \rightarrow 0$

Console.WriteLine(scores [0]); $\rightarrow$ 80が表示される

for文 2週目 $i \rightarrow 1$

Console.WriteLine(scores [1]); $\rightarrow$ 75が表示される

for文 3週目 $i \rightarrow 2$

Console.WriteLine(scores [2]); $\rightarrow$ 90が表示される

for文と配列

組み合わせると何がいいのか？

```
int[] scores = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 }  
  
Console.WriteLine(scores[0]);  
Console.WriteLine(scores[1]);  
Console.WriteLine(scores[2]);
```

色々な使用例 -if文 数学編-

色々な使用例 -if文 数学編-

```
using System;
using System.Collections.Generic;
using System.Linq;

0 個の参照
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string input = Console.ReadLine();
        int password = int.Parse(input);

        if (password == 1234)
        {
            Console.WriteLine("一致しています");
        }
    }
}
```

色々な使用例 -if文 数学編-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();
        int speed = int.Parse(text);

        if (speed > 60)
        {
            Console.WriteLine("速度超過です");
        }
    }
}
```

色々な使用例 -if文 数学編-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();
        int day = int.Parse(text);

        if (day % 2 == 0)
        {
            Console.WriteLine("ポイント2倍デーです");
        }
    }
}
```

色々な使用例 -if文 数学編-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();
        int age = int.Parse(text);

        if (age >= 6 && age <= 12)
        {
            Console.WriteLine("子供料金です");
        }
    }
}
```

色々な使用例 -if文 文字列編-

色々な使用例 -if文 文字列編-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();

        if (text == "OPEN")
        {
            Console.WriteLine("ドアが開きました");
        }
    }
}
```

色々な使用例 -if文 文字列編-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();

        if (text.Contains("@"))
        {
            Console.WriteLine("メールアドレスの形式です");
        }
    }
}
```

色々な使用例 -if文 文字列編-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();

        if (text.StartsWith("A"))
        {
            Console.WriteLine("Aグループの商品です");
        }
    }
}
```

色々な使用例 -if文 文字列編-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();

        if (text.ToLower() == "admin")
        {
            Console.WriteLine("管理者アカウントです");
        }
    }
}
```

色々な使用例 -for 配列処理-

色々な使用例 -for文 配列処理-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();
        int[] numbers = text.Split(" ").Select(int.Parse).ToArray();

        int min = numbers[0];

        for (int i = 0; i < numbers.Length; i++)
        {
            if (numbers[i] < min)
            {
                min = numbers[i];
            }
        }
    }
}
```

色々な使用例 -for文 配列処理-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();
        int[] numbers = text.Split(" ").Select(int.Parse).ToArray();

        int max = numbers[0];

        for (int i = 0; i < numbers.Length; i++)
        {
            if (numbers[i] > max)
            {
                max = numbers[i];
            }
        }
    }
}
```

色々な使用例 -for文 配列処理-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();
        int[] numbers = text.Split(" ").Select(int.Parse).ToArray();

        int count = 0;

        for (int i = 0; i < numbers.Length; i++)
        {
            if (numbers[i] % 2 == 0)
            {
                count++;
            }
        }
    }
}
```

色々な使用例 -for文 配列処理-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();
        int[] prices = text.Split(" ").Select(int.Parse).ToArray();

        int total = 0;

        for (int i = 0; i < prices.Length; i++)
        {
            total += prices[i];
        }
    }
}
```

色々な使用例 -for文 配列処理-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();
        int[] numbers = text.Split(" ").Select(int.Parse).ToArray();

        for (int i = 0; i < numbers.Length; i++)
        {
            for (int j = i + 1; j < numbers.Length; j++)
            {
                if (numbers[i] > numbers[j])
                {
                    int temp = numbers[i];
                    numbers[i] = numbers[j];
                    numbers[j] = temp;
                }
            }
        }
    }
}
```

色々な使用例 -for文 配列処理-

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();
        int[] numbers = text.Split(" ").Select(int.Parse).ToArray();

        for (int i = 0; i < numbers.Length; i++)
        {
            for (int j = i + 1; j < numbers.Length; j++)
            {
                if (numbers[i] > numbers[j])
                {
                    int temp = numbers[i];
                    numbers[i] = numbers[j];
                    numbers[j] = temp;
                }
            }
        }
    }
}
```